Assessment 04 - Block ciphers and modes of operation

Instructions
The assessment must be developed individually and submitted via Blackboard by **23/11/2026**. The submission should be a *zip* file containing all developed code and a short PDF report describing the code and discussing the results.

This assessment takes into account the use of Python and the Cryptography library. Installation instructions can be found here. However, you can choose a language and library of your preference.

All questions share the scenario of *Assessment 03*: the *Programmable Logic Controllers* (PLC) of a water treatment plant exchange commands with a control centre. From the public documentation of the PLCs, the commands follow the fixed format `PLC=<id>;CMD=SET;NAOH_PPM=<value>;`, where `<id>` has two digits and `<value>` has five digits. Ciphertexts are represented in hexadecimal. Whenever a key is provided, it is given **only so that you can check your attack**; your solution must **not** use the key.

Activities

Q1: After the audit of *Assessment 03*, the administration replaced the *ChaCha20* stream cipher with the block cipher **AES-128-CTR** (counter mode), keeping a secret key shared between the control centre and the PLCs and a unique nonce/counter block per message. You can use the implementation available in the Cryptography library (see more here). The 16-byte nonce is the full initial counter block. The ciphertext below sets the sodium hydroxide concentration to `00100` ppm.

Q1.a: Without knowing the key, produce a new ciphertext that, when decrypted by the PLC, sets the sodium hydroxide concentration to `11100` ppm.
Q1.b: The administration argues that moving from *ChaCha20* to *AES-128-CTR* "fixes the malleability problem" of *Assessment 03*. Discuss whether this migration protects against the same attack and justify your answer.

```
Nonce:
0x00000001a5f3c90e7b14d2660048ff21
Ciphertext:
0x51073653f3851c51af06c462d293862e93941666f8d561dbefdcc89cd972
Key:
0x8b2d1f0a77c4e5519903aab4cde61277
```

Q2: To "simplify the firmware", a technician reconfigured the PLCs to use **AES-128-ECB**. You intercepted the four ciphertexts below, each the encryption of one command (padded to a multiple of the block size) under the same key. Without knowing the key and **without decrypting** the messages, propose and implement a strategy that lets you determine which of the captured commands are identical to each other, and explain in general what information *AES-128-ECB* leaks about the plaintext. Discuss why this makes ECB unsuitable for protecting the PLC traffic.

```
Ciphertext 1:
0xb3f54ffd243d5764117c2751a130813068e1caf62ab7e794b90583176eeb9fc6542b191dd3114c5cdf9f06c11bbc31e0a2914503499340c978b69b612fcd6a6a
Ciphertext 2:
0xa22cbbd3231e96c2e1300049d5ca045abbd343ee4ac57cbb1110f709dfeddd19143bf488dccb3bcf42644f665d08c99f57d3dfc3b1d11f4f952474ae36f72917
Ciphertext 3:
0xb3f54ffd243d5764117c2751a130813068e1caf62ab7e794b90583176eeb9fc6542b191dd3114c5cdf9f06c11bbc31e0a2914503499340c978b69b612fcd6a6a
Ciphertext 4:
0x456437c5194fde6cbcd694083dec9478d26e0d34661e257da479f73c4952885c577dd28d11e6d4c26e7de22affd8b8dbbb3b4fd998882afc390432cfb1e99781
```

Q3: In another variant, the control centre encrypts each outgoing command with **AES-128-CBC** (see more here). The *initialization vector* (IV) is a 128-bit counter that is incremented by one for each message and transmitted in clear, so you always know the IV that the control centre will use for the next message. You can also induce the control centre to encrypt and send a command of your choosing (a chosen-plaintext capability).

You captured a command whose first 16-byte plaintext block is secret. You only know it has the format above, so the unknown part of that first block is the two-digit `<id>`. Below are the IV `iv0` and the first ciphertext block `c0` of the captured message, and the IV `iv1` that the control centre will use for your next (injected) message. Propose and implement a strategy that recovers the `<id>` of the captured command **without decrypting it and without using the key**. Afterwards, explain why using a random, unpredictable IV would prevent this attack, while a unique-but-predictable IV does not.

Obs. Since you do not have a live control centre, you may use the key below **only to emulate the encryption oracle** (i.e., to answer your chosen-plaintext queries) and to verify your result; the logic of the attack must not rely on knowing the key.

```
iv0 (IV of the captured message):
0x00000000000000000000000000001000
c0  (first ciphertext block of the captured message):
0x8e00a77c9519e662e83144b473edb147
iv1 (IV used for your next injected message):
0x00000000000000000000000000001001
Key (oracle emulation / verification only):
0xc0ffee1122334455aabbccddeeff0011
```

Q4: Following the lessons of *Q1*–*Q3*, the plant finally adopts **AES-128-GCM**, an authenticated mode (see more here), with a secret key, a unique 12-byte nonce per message, and a 16-byte authentication *tag* transmitted with each ciphertext. The values below again encrypt the command that sets the concentration to `00100` ppm.

Q4.a: Repeat the strategy of *Q1.a* (flip the ciphertext bytes of the `<value>` field to obtain `11100`) and show what happens when the PLC tries to decrypt and verify the forged message. Explain why *AES-128-GCM* defeats the attack that worked against *AES-128-CTR*.
Q4.b: *AES-128-GCM* is only secure if the nonce is never reused with the same key. Explain what an attacker gains if two different messages are encrypted under the **same** (key, nonce) pair, both in terms of confidentiality and in terms of the authentication guarantee.

```
Nonce:
0xa0acc5b086bb58000c5b0d31
Ciphertext:
0x8fc0cc2e6567e77bcfaad7c8363ae39b2db91d4df9aa5acb75dce650cfe6
Tag:
0xa903a1a5b290dfe9d41c2634deed9cb1
Key:
0x3a1c9f44db2e7780015566aabbcc9911
```